Bird Species Classification with a Custom CNN and Transfer Learning from MobileNet and VGG16

Anil Kumar
Department of Mechanical Engineering
Indian Institute of Technology Ropar
Punjab, India
2019mem1031@iitrpr.ac.in

Sandeep Singh
Department of Mechanical Engineering
Indian Institute of Technology Ropar
Punjab, India
2019mem1035@iitrpr.ac.in

***Abstract*—We classify photographs of 525 bird species and compare a 13-layer convolutional neural network (CNN) trained from scratch with three transfer learning set-ups: a MobileNet fine-tuned from ImageNet weights, a VGG16 used as a frozen feature extractor with a new fully connected head, and the same VGG16 with its last convolution block also trained. Birds are hard to tell apart in photographs because lighting and background vary, foliage hides part of the bird, and related species look alike. All four models are trained on the same split with the same augmentation, label smoothing and stopping rule, each three times with different random seeds, and are evaluated on unaltered test images. Besides accuracy we report macro-averaged precision, recall and F1 score and the F1 score of every species. The fine-tuned MobileNet reaches 99.39 ± 0.08% test accuracy and a macro F1 of 0.994. Training the last VGG16 block raises its accuracy from 96.81 ± 0.17% to 98.26 ± 0.18%, and the CNN trained from scratch reaches 96.99 ± 0.31%. MobileNet has 3.7 million trainable parameters, against 394 million in the VGG16 head.**

***Index Terms*—image classification, convolutional neural network, transfer learning, MobileNet, VGG16, bird species**

I. Introduction

A computer receives a colour photograph as a three-dimensional array of numbers: height, width, and one channel each for red, green and blue. Classifying the image means finding a function from that array to a label that also gives the right answer on images it has not seen.

Convolutional neural networks [1] learn such functions directly from pixels. Early layers respond to edges and blobs, and later layers combine these into parts and whole objects. Since AlexNet won the ImageNet challenge in 2012 [2], CNNs have been the default model for image classification [3]. Their weakness is data: a deep network trained from scratch needs many labelled images per class. Transfer learning reduces this need. A network is first trained on a large dataset such as ImageNet [4], and its learned filters are then reused for a new task with less data [5].

Bird species classification is a fine-grained problem. Within a genus, two species may differ only in a wing bar or the colour of the throat, while pictures of the same species can differ a great deal in pose, light and background. Males and females of one species often look different.

This paper compares two strategies on all 525 species of a public bird dataset. In the first we design and train a CNN from scratch. In the second we start from networks pretrained on ImageNet: MobileNet [6], which we fine-tune end to end, and VGG16 [7], whose convolutional layers we freeze, either all of them or all but the last block. We report accuracy and also macro-averaged precision, recall and F1 score, because with 525 classes a model can reach a high overall accuracy while failing on some classes entirely.

A first version of this study was done in 2020 on a 185-species version of the dataset. Its models were not all trained on the same classes, and its validation and test images were passed through random augmentation before being scored. We rebuilt the experiment so that all models share one data split, one training procedure and one evaluation code path, and we trained each model with three seeds. Section VIII compares the new results with the 2020 ones. The code, training logs and per-run results are at https://github.com/a-k-sangwan/bird-species-classification.

II. Related work

Tayal [8] used AlexNet, pretrained on ImageNet, to extract features from bird images and classified them with a support vector machine (SVM). Niemi and Tanttu [9] trained a CNN for bird identification and enlarged the training set by rotating images and shifting them to different colour temperatures. Roslan et al. [10] used hand-crafted colour features, the mean, standard deviation and skewness of each RGB channel, nine values per image, as input to an SVM. Our work differs in comparing a CNN trained from scratch with several transfer learning set-ups on the same dataset.

III. Dataset

We used the bird species dataset compiled by Piosenka [11]. In 2020 it had 185 species. The Kaggle page has since been removed, so we used a public copy of its final 525-species version on the Hugging Face Hub [12], released under a CC0 licence. It has 84 635 training images, between 130 and 263 per species (median 158), and 2 625 validation and 2 625 test images, five per species. All images are 224×224 RGB JPEG files, and each was cropped so that the bird fills at least half of the frame. Fig. 1 shows examples.

The dataset's author reported that more than 75% of the training images show male birds. In species where the sexes



Fig. 1. Training images from eight of the 525 species.

look different, a model trained on these data sees few females. We did not measure accuracy separately for males and females.

IV. Background

A. Building blocks of a CNN

A convolution layer slides a small kernel K over the input I and, at each position, sums the element-wise product of the kernel and the patch beneath it. For a two-dimensional input the output feature map is

$$S(i,j) = \sum_m \sum_n I(i+m,\ j+n)\, K(m,n). \tag{1}$$

(Deep learning libraries implement this cross-correlation and call it convolution.) A layer with N kernels produces N feature maps.

Each convolution is followed by a non-linear activation. We use the rectified linear unit, $\mathrm{ReLU}(z) = \max(0, z)$, which passes positive inputs unchanged and sets negative ones to zero.

A feature map records where each feature occurs, so a small shift of the input changes it. A pooling layer down-samples the map, for example by taking the maximum over each 2×2 window with stride 2. This halves the height and width and makes the output change little when the input moves by a pixel or two [5].

Batch normalization [13] normalizes each activation over a mini-batch $\mathcal{B}$ and then rescales it with learned parameters γ and β:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta. \tag{2}$$

It lets the network train with higher learning rates and makes training less sensitive to initialization.

Dropout [14] sets a random fraction of the units of a layer to zero at each training step. Each step therefore trains a different thinned network, which reduces overfitting. At test time all units are kept.

After the last convolution or pooling layer, the feature maps are flattened into a vector and passed to one or more fully connected (dense) layers, in which every unit is connected to every unit of the previous layer. The final dense layer has one unit per class and a softmax activation, so its outputs are class probabilities.

B. Depthwise separable convolution

MobileNet [6] replaces standard convolutions with depthwise separable convolutions. A standard convolution filters the input channels and combines them into new features in one step. A depthwise separable convolution splits this into two layers: a depthwise convolution that applies one $D_K \times D_K$ filter to each input channel separately, and a pointwise 1×1 convolution that forms linear combinations of the depthwise outputs. For an input with M channels, N output channels and an output feature map of size $D_F \times D_F$, the cost of a standard convolution is $D_K^2 M N D_F^2$ multiply-adds, and the cost of the separable version is $D_K^2 M D_F^2 + M N D_F^2$. The ratio is

$$\frac{D_K^2 M D_F^2 + M N D_F^2}{D_K^2 M N D_F^2} = \frac{1}{N} + \frac{1}{D_K^2}, \tag{3}$$

so with 3×3 kernels the separable layer needs 8 to 9 times fewer operations. On ImageNet, MobileNet is about one percentage point less accurate than VGG16 while having 32 times fewer parameters and needing 27 times fewer operations [6].

V. Models

A. CNN trained from scratch

Our CNN has 13 convolution layers, one hidden dense layer and a softmax output layer (Table I). Every convolution uses 3×3 kernels and "same" padding, so it keeps the height and width of its input, and is followed by batch normalization and ReLU. The first twelve convolutions are arranged in six blocks of two. Each block ends in 2×2 max pooling with stride 2, and the number of filters doubles from block to block, from 8 to 256. A thirteenth convolution with 512 filters and a final pooling layer reduce the feature map to $1 \times 1 \times 512$. This vector is batch normalized and passed through dropout, a 512-unit dense layer with its own batch normalization, ReLU and dropout, and a 525-unit softmax layer. The model has 2.9 million trainable parameters.

B. Transfer learning with MobileNet

We use the standard MobileNet body [6], initialized with ImageNet weights. It has 28 layers if depthwise and pointwise convolutions are counted separately, and every layer is followed by batch normalization and ReLU. For a $224 \times 224 \times 3$ input the body produces a $7 \times 7 \times 1024$ feature map. We made two changes at the top. The original 7×7 global average pooling is replaced by 7×7 max pooling, and the 1000-class ImageNet classifier is replaced by batch normalization and a 525-unit softmax layer. All layers, including the pretrained ones, are trained. The model has 3.7 million trainable parameters.

TABLE I
ARCHITECTURE OF THE CNN TRAINED FROM SCRATCH

Stage	Layers	Output size
Input		$224 \times 224 \times 3$
Block 1	2× conv 8, max pool	$112 \times 112 \times 8$
Block 2	2× conv 16, max pool	$56 \times 56 \times 16$
Block 3	2× conv 32, max pool	$28 \times 28 \times 32$
Block 4	2× conv 64, max pool	$14 \times 14 \times 64$
Block 5	2× conv 128, max pool	$7 \times 7 \times 128$
Block 6	2× conv 256, max pool	$3 \times 3 \times 256$
Block 7	1× conv 512, max pool	$1 \times 1 \times 512$
Head	flatten, BN, dropout 0.4	512
	dense 512, BN, ReLU, dropout 0.5	512
	dense 525, softmax	525

Each conv layer uses 3×3 kernels with same padding and is followed by batch normalization (BN) and ReLU. Max pooling is 2×2 with stride 2.

TABLE II
CLASSIFIER HEAD ADDED TO THE VGG16 BODY

Layer	Output size
VGG16 conv blocks 1–5	$7 \times 7 \times 512$
Flatten	25 088
Dense, BN, ReLU, dropout 0.5	10 000
Dense, BN, ReLU, dropout 0.4	10 000
Dense, ReLU, dropout 0.3	4 000
Dense, ReLU	700
Dense, softmax	525

C. Transfer learning with VGG16

VGG16 [7] has 13 convolution layers with 3×3 kernels in five blocks, each ending in max pooling, followed by three dense layers. We keep the five convolution blocks with their ImageNet weights and freeze them, so they act as a fixed feature extractor [5]. Their $7 \times 7 \times 512$ output is flattened to a 25 088-element vector and fed to the new head in Table II. Only the head is trained. It has 394 million parameters, most of them in the first two dense layers.

A frozen body cannot adapt its filters to birds. To measure what this costs, we also train a second version with the same head in which the three convolution layers of block 5 are trained as well and blocks 1 to 4 stay frozen. This adds 7.1 million trainable parameters. We call the two versions VGG16 (frozen) and VGG16 (block 5 tuned).

VI. TRAINING AND EVALUATION

The models were built in Keras 3.15 on TensorFlow 2.21 and trained on one NVIDIA RTX 4000 Ada GPU (20 GB) with mixed-precision arithmetic and a batch size of 64. Images are read at 224×224 pixels. During training each batch is randomly flipped left to right, shifted by up to 10% of the image size in each direction and zoomed by up to 10%. These are the 2020 augmentation settings without the vertical flip, since no test image shows a bird upside down. Validation and test images are not augmented. Each model then applies the input scaling its weights expect: division by 255 for the CNN, scaling to $[-1, 1]$ for MobileNet, and the ImageNet channel-mean subtraction in BGR order for VGG16.

TABLE III
TRAINING SUMMARY, MEAN ± STANDARD DEVIATION OVER THREE SEEDS. ACCURACY (%) IS FOR THE EPOCH WITH THE LOWEST VALIDATION LOSS.

Model	Params (M)	Epochs	Time (min)	Train	Val.
CNN (scratch)	2.9	77 ± 6	157 ± 10	94.48 ± 0.28	93.59 ± 0.58
MobileNet	3.7	50	230 ± 1	99.95 ± 0.01	98.22 ± 0.13
VGG16 (frozen)	394.1	40	145 ± 2	95.10 ± 0.13	94.69 ± 0.12
VGG16 (block 5)	401.2	40	125 ± 2	99.22 ± 0.12	96.33 ± 0.12

Params counts trainable parameters. Training accuracy is measured on augmented images and validation accuracy on single, unmirrored images.

We kept the 2020 optimizers: Adamax with a learning rate of 0.006 for the CNN and MobileNet, and stochastic gradient descent with a learning rate of 0.01 for both VGG16 versions. Three things were added to the 2020 recipe, the same for every model. The loss is cross-entropy with label smoothing of 0.1, so the target gives the true species a probability of 0.9 plus its share of the remaining 0.1. The learning rate rises linearly from zero to its set value over the first two epochs, to keep large early updates from disturbing the pretrained weights. At test time each image is classified twice, as given and mirrored left to right, and the two softmax outputs are averaged.

After the warm-up, the learning rate is halved when the validation loss has not improved for three epochs, and training stops when it has not improved for eight epochs or when the epoch limit is reached (80 for the CNN, 50 for MobileNet, 40 for both VGG16 versions). The weights with the lowest validation loss are kept and used on the test set. Each model was trained three times, with seeds 42, 43 and 44. The seed sets the weight initialization, the dropout masks, the augmentation and the order of the training batches. We report the mean and standard deviation over the three runs.

Every metric is computed from one set of predictions per run. We report accuracy, top-5 accuracy, and precision, recall and F1 score for each class c,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{4}$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \tag{5}$$

averaged with equal weight over classes (macro average). Each test class has the same number of images, so macro-averaged recall equals accuracy.

VII. RESULTS

Table III summarizes training, and Table IV gives the test metrics, which Fig. 3 also plots. Fig. 2 shows the training curves of the seed-42 runs.

The fine-tuned MobileNet is the best model on every measure. It misclassifies 14, 16 and 18 of the 2 625 test images in its three runs, for a mean accuracy of 99.39% and a macro F1 of 0.994. Its validation accuracy passes 90% at epoch 5 or 6 and 95% at epoch 9 or 10. Its best epoch was 45, 47 and 48 of 50, so its validation loss was still falling near the epoch limit. Its training accuracy (99.95%) is 1.7 points above its validation accuracy.

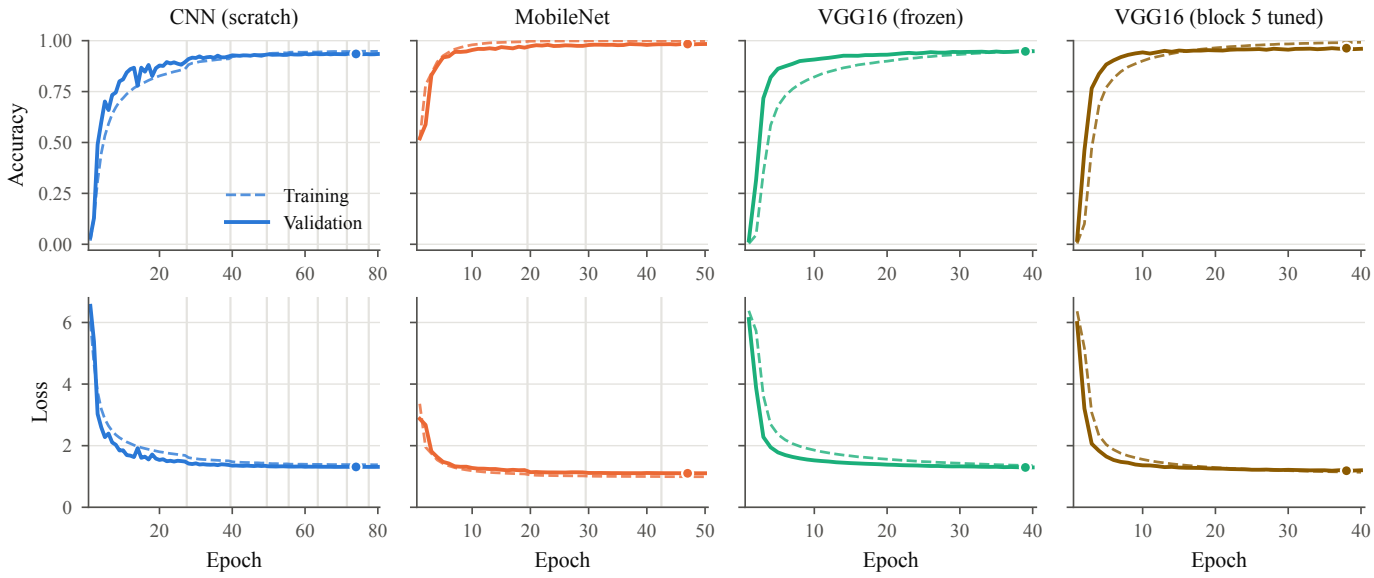


Fig. 2. Accuracy (top) and loss (bottom) on the training set (dashed) and validation set (solid) for the seed-42 runs. The dot marks the epoch with the lowest validation loss, whose weights are used on the test set. Vertical grey lines mark epochs at which the learning rate was halved. The loss includes label smoothing, so it does not fall to zero.

TABLE IV
TEST METRICS ON 2 625 IMAGES OF 525 SPECIES, MEAN ± STANDARD DEVIATION OVER THREE SEEDS

Model	Accuracy (%)	Single image (%)	Top-5 (%)	Macro F1
CNN (scratch)	96.99 ± 0.31	96.03 ± 0.34	99.35 ± 0.04	0.969 ± 0.003
MobileNet	99.39 ± 0.08	99.30 ± 0.13	99.90 ± 0.08	0.994 ± 0.001
VGG16 (frozen)	96.81 ± 0.17	96.62 ± 0.28	99.75 ± 0.15	0.968 ± 0.002
VGG16 (block 5)	98.26 ± 0.18	98.02 ± 0.10	99.87 ± 0.04	0.982 ± 0.002

Accuracy, top-5 and F1 average the predictions for each image and its mirror image. Single image is the accuracy without this averaging.

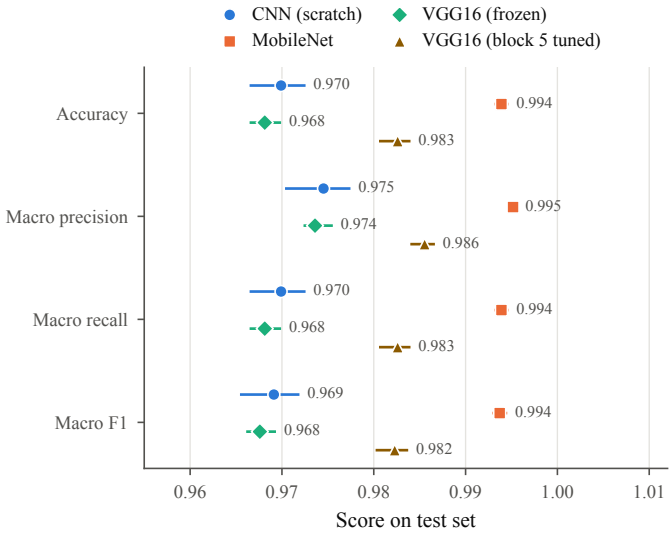


Fig. 3. Test accuracy and macro-averaged precision, recall and F1 score. Marks are the mean of three seeds and bars span the lowest to the highest seed.

Training block 5 of VGG16 cuts its test errors from 84 to 46 on average and raises its accuracy from 96.81% to 98.26%. The difference is more than eight times the standard deviation of either version. The frozen VGG16 fits its training data no better than its validation data (95.10% against 94.69%). Its learning rate was never halved, because its validation loss kept improving, and its best epoch was the 39th or 40th of 40. The tuned version fits its training data much more closely (99.22%) and reaches 96.33% on validation, a gap of 2.9 points.

The CNN trained from scratch reaches 96.99 ± 0.31%, the same as the frozen VGG16 to within the seed-to-seed spread. Its validation accuracy passes 90% at epoch 22 to 28 and peaks at 93.2% to 94.4%. Its learning rate was halved seven or eight times. Two runs used all 80 epochs, with best epochs 73 and 74, and the third stopped early at epoch 70. After epoch 5 its validation accuracy is never more than 1.8 points below its training accuracy, so it is not limited by overfitting. Its top-5 accuracy is 99.35%: when it is wrong, the correct species is usually among its next four guesses.

Averaging each test prediction with that of the mirrored image helps every model, by 0.96 points for the CNN and by 0.09 to 0.24 points for the three pretrained models. The single-image accuracies in Table IV put the models in the same order.

For all four models the validation accuracy is lower than the single-image test accuracy, by 1.1 points (MobileNet) to 2.4 points (CNN). Validation and test sets have the same size and five images per species. We did not look into the cause of the difference.

The CNN is the fastest model at test time, at 0.4 ms per image, against 0.8 ms for MobileNet and 2.0 ms for both VGG16 versions, in batches of 64 on the GPU.

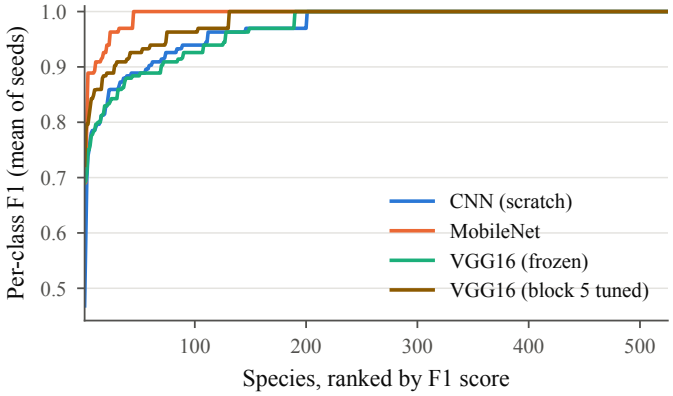


Fig. 4. F1 score of each of the 525 species, averaged over three seeds and sorted from lowest to highest for each model.

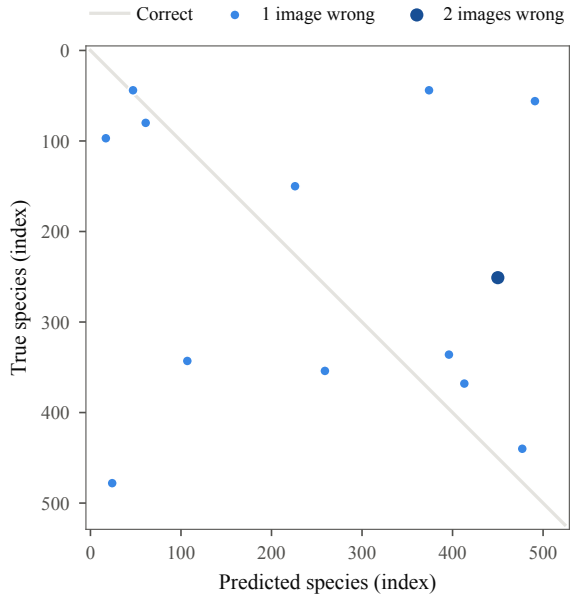


Fig. 5. Errors of MobileNet (seed 42) on the test set, drawn as a confusion matrix. Each point is a (true, predicted) pair of different species; the larger point is the one pair with two of its five test images misclassified. All other cells off the grey diagonal are zero.

Fig. 4 shows how the errors are spread over species. With five test images per species, a species' F1 score in one run can take only a few values, which gives the curves their steps. MobileNet classifies 481 of the 525 species perfectly in all three runs and has two species with a mean F1 below 0.8: the grey partridge and the Austral canastero, both at 0.69. VGG16 with block 5 tuned has 395 such species and four below 0.8, the frozen VGG16 336 and 12, and the CNN 325 and 14. The CNN's worst species is Abbott's booby, with a mean F1 of 0.47. Twenty-nine species have at least one error in some run of every model.

The seed-42 MobileNet makes 14 errors, spread over 13 cells of its confusion matrix (Fig. 5). Only one pair of species is confused twice: two grey partridges are taken for snow partridges. Most of the other errors are between related species



Fig. 6. Four MobileNet (seed 42) errors. Top: the test image, labelled with its true species. Bottom: a training image of the species MobileNet predicted. The count gives the number of test images with this error.

TABLE V
TEST ACCURACY (%) IN 2020, IN A SINGLE-SEED RERUN ON 200 SPECIES WITH THE 2020 RECIPE, AND IN THIS WORK

Model	2020	Rerun	This work
CNN (scratch)	89.7	94.4	97.0
MobileNet	98.4	98.5	99.4
VGG16 (frozen)	95	96.4	96.8
VGG16 (block 5)	–	–	98.3
Species	185 (VGG16: 200)	200	525
Seeds	1	1	3

of similar build: a black-necked stilt taken for a banded stilt, an orange-breasted trogon for a red-naped trogon, a malachite kingfisher for a pygmy kingfisher, a Baltimore oriole for a Venezuelan troupial (both black and orange icterids), and an Austral canastero for an Azara's spinetail, both small brown ovenbirds. Fig. 6 shows four of these errors.

VIII. Comparison with the earlier runs

Table V sets the 2020 test results beside two later sets. The middle column comes from a run we made on 200 randomly chosen species with the 2020 recipe (no label smoothing, no warm-up, no test-time mirroring) and one seed, as a step between the 2020 code and this work. The 2020 CNN and MobileNet were trained on 185 species and the 2020 VGG16 on 200, so no two columns use the same set of classes.

Three differences in the 2020 code affect its numbers. First, the CNN and MobileNet notebooks used one augmenting data generator for training, validation and test images, so every evaluation scored a different random shift, zoom and flip of the test set. Keras' built-in evaluation of the 2020 CNN gave 93.7% test accuracy and the per-image classification report 89.7%. Both were computed on separately augmented copies of the test set, which may explain the four-point difference. Second, the 2020 VGG16 was trained without augmentation and normalized each image by its own mean and standard deviation, while its ImageNet weights expect the channel-

mean subtraction we now use. It fitted its training data almost perfectly (99.5%) and reached 92.6% on validation. With augmentation and matching preprocessing, its training and validation accuracies are now within half a point of each other. Third, the 2020 CNN used a hand-written learning-rate callback and ran for 73 epochs in about six hours.

The CNN gains the most between the rerun and this work, 2.6 points, even though the number of species grew from 200 to 525. The test-time mirroring accounts for about one point of this. We did not separate the effects of label smoothing, warm-up and the change of species.

IX. Discussion

Both pretrained models start from filters learned on more than a million ImageNet images, which include many bird classes. The CNN trained from scratch has to learn its filters from about 160 images per species. This explains most of the gap between MobileNet and our CNN: the CNN makes about five times as many test errors (79 against 16 on average).

The two VGG16 versions show that the frozen body, more than the size of the head, limited the frozen VGG16. Training block 5 alone removed 45% of its test errors with the same head. Even so, VGG16 with block 5 tuned makes almost three times as many errors as MobileNet, which trains all of its layers and has a hundred times fewer parameters. We did not train all VGG16 layers, so we cannot say whether a fully fine-tuned VGG16 would match MobileNet.

Our results have four limitations. The test set has five images per species, so one wrong prediction changes a species' recall by 0.2 and the per-species scores in Fig. 4 rest on very few samples. Three seeds give a rough measure of run-to-run spread, not a confidence interval. We did not check whether any test image is a near-duplicate of a training image, which would raise all test scores. The training data contain few images of female birds, and we did not measure how well any of the models recognize them.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

[3] M. Xin and Y. Wang, "Research on image classification model based on deep convolution neural network," *EURASIP Journal on Image and Video Processing*, vol. 2019, p. 40, 2019.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[5] S. Tammina, "Transfer learning using VGG-16 with deep convolutional neural network for classifying images," *International Journal of Scientific and Research Publications*, vol. 9, no. 10, pp. 143–150, 2019.

[6] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017.

[7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014.

[8] M. Tayal, "Bird identification by image recognition," 2018.

[9] J. Niemi and J. T. Tanttu, "Deep learning case study for automatic bird identification," *Applied Sciences*, vol. 8, no. 11, p. 2089, 2018.

[10] R. Roslan *et al.*, "Color-based bird image classification using support vector machine," 2017.

[11] G. Piosenka, "Bird species dataset," Kaggle, 2020, https://www.kaggle.com/gpiosenka/100-bird-species.

[12] chriamue, "Bird species dataset (copy of the kaggle 525-species version, cc0)," Hugging Face Hub, https://huggingface.co/datasets/chriamue/bird-species-dataset.

[13] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 448–456.

[14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.